

respostas_av1

VHDL solutions, parts 1 to 5 and extra activity

Part 1

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;

ENTITY part1 IS
PORT (SW : IN STD_LOGIC_VECTOR(9 DOWNTO 0);
      LEDG : OUT STD_LOGIC_VECTOR(9 DOWNTO 0));
END part1;

ARCHITECTURE hardware OF part1 IS
BEGIN LEDG <= SW;
END hardware;
```

Part 2

```
library ieee;
use ieee.std_logic_1164.all;

entity part2 is
    port (
        SW    : in  std_logic_vector(9 downto 0);
        LEDG  : out std_logic_vector(9 downto 0)
    );
end part2;

architecture rtl of part2 is
begin
    LEDG(3 downto 0) <= SW(3 downto 0) when SW(9) = '0' else SW(7 downto 4);
    LEDG(8 downto 4) <= (others => '0');
    LEDG(9)         <= SW(9);
end rtl;
```

Part 3

```
library ieee;
use ieee.std_logic_1164.all;

entity part3 is
    port (
        SW    : in  std_logic_vector(9 downto 0);
        LEDG  : out std_logic_vector(9 downto 0)
    );
end part3;

architecture behavioral of part3 is
begin
    process (SW)
    begin
        LEDG <= (others => '0');

        case SW(9 downto 8) is
            when "00"  => LEDG(1 downto 0) <= SW(1 downto 0);
            when "01"  => LEDG(1 downto 0) <= SW(3 downto 2);
            when "10"  => LEDG(1 downto 0) <= SW(5 downto 4);
            when "11"  => LEDG(1 downto 0) <= SW(7 downto 6);
            when others => null;
        end case;
    end process;
end behavioral;
```

Part 4

```
library ieee;
use ieee.std_logic_1164.all;

entity part4 is
    port (
        SW    : in  std_logic_vector(1 downto 0);
        LEDG  : out std_logic_vector(9 downto 0);
        HEX0  : out std_logic_vector(0 to 6)
    );
end part4;

architecture rtl of part4 is
    constant CHAR_D    : std_logic_vector(0 to 6) := "1000010";
    constant CHAR_E    : std_logic_vector(0 to 6) := "0110000";
    constant CHAR_0    : std_logic_vector(0 to 6) := "0000001";
    constant CHAR_BLANK : std_logic_vector(0 to 6) := "1111111";
begin
    LEDG(1 downto 0) <= SW;
    LEDG(9 downto 2) <= (others => '0');
    with SW select
        HEX0 <= CHAR_D    when "00",
              CHAR_E    when "01",
              CHAR_0    when "10",
              CHAR_BLANK when others;
end rtl;
```

Part 5

```
library ieee;
use ieee.std_logic_1164.all;

-- Multiplexador 4 para 1 de dois bits, equivalente ao circuito da Parte 3.
entity mux4_2bit is
    port (
        S      : in  std_logic_vector(1 downto 0);
        U, V, W, X : in  std_logic_vector(1 downto 0);
        M      : out std_logic_vector(1 downto 0)
    );
end mux4_2bit;

architecture behavioral of mux4_2bit is
begin
    process (S, U, V, W, X)
    begin
        case S is
            when "00" => M <= U;
            when "01" => M <= V;
            when "10" => M <= W;
            when "11" => M <= X;
            when others => M <= (others => '0');
        end case;
    end process;
end behavioral;

library ieee;
use ieee.std_logic_1164.all;

-- Decodificador da Parte 4. Os segmentos da DE0 sao ativos em nivel 0.
entity char_7seg is
    port (
        C      : in  std_logic_vector(1 downto 0);
        DISPLAY : out std_logic_vector(0 to 6)
    );
end char_7seg;

architecture rtl of char_7seg is
    constant CHAR_D : std_logic_vector(0 to 6) := "1000010";
    constant CHAR_E : std_logic_vector(0 to 6) := "0110000";
    constant CHAR_0  : std_logic_vector(0 to 6) := "0000001";
    constant CHAR_BLANK : std_logic_vector(0 to 6) := "1111111";
begin
    with C select
        DISPLAY <= CHAR_D    when "00",
                 CHAR_E      when "01",
                 CHAR_0       when "10",
                 CHAR_BLANK   when others;
end rtl;

library ieee;
use ieee.std_logic_1164.all;

entity part5 is
    port (
        SW      : in  std_logic_vector(9 downto 0);
        LEDG    : out std_logic_vector(9 downto 0);
        HEX3, HEX2, HEX1, HEX0 : out std_logic_vector(0 to 6)
    );
end part5;

architecture structural of part5 is
    signal SELECTOR : std_logic_vector(1 downto 0);
```

```

    signal CH0, CH1, CH2, CH3          : std_logic_vector(1 downto 0);
    signal H3_CHAR, H2_CHAR, H1_CHAR, H0_CHAR : std_logic_vector(1 downto 0);
begin
    LEDG <= SW;

    SELECTOR <= SW(9 downto 8);
    CH0 <= SW(7 downto 6);
    CH1 <= SW(5 downto 4);
    CH2 <= SW(3 downto 2);
    CH3 <= SW(1 downto 0);

    MUX_HEX3 : entity work.mux4_2bit(behavioral)
        port map (SELECTOR, CH0, CH1, CH2, CH3, H3_CHAR);

    MUX_HEX2 : entity work.mux4_2bit(behavioral)
        port map (SELECTOR, CH1, CH2, CH3, CH0, H2_CHAR);

    MUX_HEX1 : entity work.mux4_2bit(behavioral)
        port map (SELECTOR, CH2, CH3, CH0, CH1, H1_CHAR);

    MUX_HEX0 : entity work.mux4_2bit(behavioral)
        port map (SELECTOR, CH3, CH0, CH1, CH2, H0_CHAR);

    DEC_HEX3 : entity work.char_7seg(rtl) port map (H3_CHAR, HEX3);
    DEC_HEX2 : entity work.char_7seg(rtl) port map (H2_CHAR, HEX2);
    DEC_HEX1 : entity work.char_7seg(rtl) port map (H1_CHAR, HEX1);
    DEC_HEX0 : entity work.char_7seg(rtl) port map (H0_CHAR, HEX0);
end structural;

```

Extra Activity

```
library ieee;
use ieee.std_logic_1164.all;

-- Multiplexador 3 para 1 de dois bits pedido na Atividade Extra.
entity mux3_2bit is
    port (
        S      : in  std_logic_vector(1 downto 0);
        U, V, W : in  std_logic_vector(1 downto 0);
        M      : out std_logic_vector(1 downto 0)
    );
end mux3_2bit;

architecture behavioral of mux3_2bit is
begin
    process (S, U, V, W)
    begin
        case S is
            when "00"  => M <= U;
            when "01"  => M <= V;
            when "10"  => M <= W;
            when others => M <= U;
        end case;
    end process;
end behavioral;

library ieee;
use ieee.std_logic_1164.all;

-- Mesmo decodificador da Parte 4, para os displays ativos em nivel baixo.
entity char_7seg is
    port (
        C      : in  std_logic_vector(1 downto 0);
        DISPLAY : out std_logic_vector(0 to 6)
    );
end char_7seg;

architecture rtl of char_7seg is
    constant CHAR_D : std_logic_vector(0 to 6) := "1000010";
    constant CHAR_E : std_logic_vector(0 to 6) := "0110000";
    constant CHAR_0  : std_logic_vector(0 to 6) := "0000001";
    constant CHAR_BLANK : std_logic_vector(0 to 6) := "1111111";
begin
    with C select
        DISPLAY <= CHAR_D    when "00",
                 CHAR_E      when "01",
                 CHAR_0       when "10",
                 CHAR_BLANK   when others;
end rtl;

library ieee;
use ieee.std_logic_1164.all;

entity extra is
    port (
        SW      : in  std_logic_vector(9 downto 0);
        LEDG    : out std_logic_vector(9 downto 0);
        HEX2, HEX1, HEX0 : out std_logic_vector(0 to 6)
    );
end extra;

architecture structural of extra is
    signal SELECTOR : std_logic_vector(1 downto 0);
    signal CH0, CH1, CH2 : std_logic_vector(1 downto 0);
```

```

    signal H2_CHAR, H1_CHAR, H0_CHAR : std_logic_vector(1 downto 0);
begin
    LEDG <= SW;

    SELECTOR <= SW(9 downto 8);
    CH0 <= SW(5 downto 4);
    CH1 <= SW(3 downto 2);
    CH2 <= SW(1 downto 0);

    MUX_HEX2 : entity work.mux3_2bit(behavioral)
        port map (SELECTOR, CH0, CH1, CH2, H2_CHAR);

    MUX_HEX1 : entity work.mux3_2bit(behavioral)
        port map (SELECTOR, CH1, CH2, CH0, H1_CHAR);

    MUX_HEX0 : entity work.mux3_2bit(behavioral)
        port map (SELECTOR, CH2, CH0, CH1, H0_CHAR);

    DEC_HEX2 : entity work.char_7seg(rtl) port map (H2_CHAR, HEX2);
    DEC_HEX1 : entity work.char_7seg(rtl) port map (H1_CHAR, HEX1);
    DEC_HEX0 : entity work.char_7seg(rtl) port map (H0_CHAR, HEX0);
end structural;

```